

Replicación y Sharding

Escalamiento horizontal, alta disponibilidad y consistencia en SD

Claudio Omar Biale

De la teoría a la práctica

- **Ya vimos:** Gossip para membresía P2P y DHT Chord para lookup distribuido.
- **Nuevo problema:** ¿Cómo almacenar datos de forma confiable en múltiples nodos?
- **Desafíos:** caídas de nodos, particiones de red, escrituras concurrentes.

La replicación da tolerancia a fallos; el sharding da escala horizontal. Juntos permiten sistemas como Dynamo, Cassandra y Bigtable.

PARTE I: Replicación

Replicación: concepto

- **Definición:** mantener múltiples copias de un dato en distintos nodos.
- **Objetivos:**
 - **Alta disponibilidad:** el servicio sigue funcionando si un nodo cae.
 - **Rendimiento:** lecturas servidas por la réplica más cercana.
 - **Tolerancia a fallos:** el sistema opera con nodos caídos.

Checkpoint: Replicar no es gratis: más copias = más almacenamiento y más mensajes de sincronización. Hay que elegir cuántas copias (N) y cómo propagar los cambios.

Modelos de replicación

- **Replicación activa (multi-master):**
 - Todas las réplicas procesan todas las operaciones.
 - Requieren operaciones determinísticas.
 - Ej: bases de datos multi-master, State Machine Replication.
- **Replicación pasiva (single-master):**
 - Un nodo primario procesa escrituras, los secundarios replican.
 - Más simple, punto único de fallo temporal.
 - Ej: PostgreSQL streaming replication, MySQL.

Estrategias de propagación

- **Síncrona**: la escritura se confirma solo cuando **todas** las réplicas actualizaron.
 - Consistencia fuerte, mayor latencia de escritura.
- **Asíncrona**: la escritura se confirma antes de propagar a réplicas.
 - Mejor rendimiento, posible inconsistencia temporal.
- **Híbrida (quórum)**: se confirma cuando un subconjunto de réplicas responde.
 - Balance entre consistencia y rendimiento.

Checkpoint: No existe replicación "mejor" en abstracto. La estrategia depende del caso de uso: ¿prefieres latencia baja o consistencia fuerte?

Topologías de replicación

Topología	Escrituras	Lecturas	Consistencia
Single-master	1 nodo	Cualquier réplica	Fuerte (sincrónica)
Multi-master	Cualquier nodo	Cualquier nodo	Eventual / conflictos
Chain replication	Líder de cadena	Cualquier nodo	Fuerte

- **Single-master**: simple, punto único de fallo.
- **Multi-master**: alta disponibilidad, requiere resolución de conflictos.
- **Chain replication**: balance entre rendimiento y consistencia.

Quórum: el modelo (N, W, R)

- N = número total de réplicas.
- W = número de confirmaciones necesarias para una escritura exitosa.
- R = número de confirmaciones necesarias para una lectura exitosa.

Propiedad fundamental: $W + R > N$ garantiza que lectura y escritura compartan al menos un nodo → lecturas siempre ven la escritura más reciente.

Checkpoint: Para N=3: (W=2, R=2) es equilibrado, tolera 1 falla. (W=3, R=1) optimiza lecturas. (W=1, R=3) optimiza escrituras pero lee de todos.

Quórum: ejemplos de configuración

N=3, W=2, R=2: tolera 1 nodo caído
N=5, W=3, R=3: tolera 2 nodos caídos
N=3, W=1, R=3: escrituras rápidas, lecturas lentas
N=3, W=3, R=1: lecturas rápidas, escrituras lentas

```
type QuorumConfig struct {  
    N int // réplicas totales  
    W int // confirmaciones de escritura  
    R int // confirmaciones de lectura  
}  
  
func (q QuorumConfig) Validar() bool {  
    return q.W + q.R > q.N  
}
```

Consistencia en sistemas replicados

- **Consistencia fuerte (linealizable)**: todos los nodos ven los mismos datos al mismo tiempo. Como si hubiera un solo nodo.
- **Consistencia secuencial**: las operaciones de cada cliente se ven en orden, pero las de distintos clientes pueden intercalarse.
- **Consistencia eventual**: si no hay nuevas escrituras, todas las réplicas convergen al mismo valor.
- **Consistencia causal**: respeta relaciones causa-efecto entre eventos.
- **Read-your-writes**: un cliente siempre ve sus propias escrituras.

Teorema CAP y PACELC

CAP (Brewer, 2000): no se puede garantizar simultáneamente:

- **C**onsistencia fuerte
- **A**lta disponibilidad
- Tolerancia a **P**articiones de red

PACELC (Abadi, 2012): extiende CAP:

- Si hay **P**artición → elegir entre **C**onsistencia o **A**vailabilidad.
- Si **E**lse (sin partición) → elegir entre **L**atencia o **C**onsistencia.

Checkpoint: PACELC explica mejor las decisiones de diseño: Dynamo elige AP+EL (alta disponibilidad, baja latencia), Spanner elige CP+EC (consistencia fuerte).

Go: Servicio de Quórum con net/rpc

Estructura de los mensajes RPC:

```
type ArgsEscritura struct {  
    Clave string  
    Valor string  
    Timestamp int64  
}  
  
type RespEscritura struct {  
    Exito bool  
    NodoID string  
}
```

Go: Servicio de Quórum con net/rpc

Servicio RPC que expone escritura y lectura con quórum. El almacenamiento y su mutex se encapsulan en `Store`, separado de la lógica RPC:

```
// Store encapsula el almacenamiento local con timestamps y su propio mutex.
type Store struct {
    Datos      map[string]string
    Versiones  map[string]int64
    mu         sync.RWMutex
}

type ServicioQuorum struct {
    NodoID string
    Store  *Store
    Pares  []string
    Config QuorumConfig
}
```

Go: Escribir con quórum

```
func (s *Store) Escribir(clave, valor string, timestamp int64) bool {
    s.mu.Lock()
    defer s.mu.Unlock()
    if ts, ok := s.Versiones[clave]; ok && timestamp < ts {
        return false
    }
    s.Datos[clave] = valor
    s.Versiones[clave] = timestamp
    return true
}

func (s *ServicioQuorum) Escribir(args ArgsEscritura, resp *RespEscritura) error {
    s.Store.Escribir(args.Clave, args.Valor, args.Timestamp)
    resp.Exito = true
    resp.NodoID = s.NodoID
    return nil
}
```

Go: Escribir con quórum

El cliente coordina el quórum:

```
func CoordinarEscritura(clave, valor string, timestamp int64, pares []string, w int) bool {
    exitos := 0
    for _, p := range pares {
        cliente, err := rpc.Dial("tcp", p)
        if err != nil {
            continue
        }
        var resp RespEscritura
        if err := cliente.Call("ServicioQuorum.Escribir",
            ArgsEscritura{Clave: clave, Valor: valor, Timestamp: timestamp}, &resp); err == nil {
            exitos++
        }
        cliente.Close()
    }
    return exitos >= w
}
```

Checkpoint: El cliente es quien decide si el quórum se cumple. El servidor solo aplica la operación. Esto se llama "coordinación descentralizada".

Go: Leer con quórum y última versión

```
type ArgsLectura struct { Clave string }
type RespLectura struct {
    Valor      string
    Timestamp  int64
    NodoID     string
    Encontrado bool
}

func (s *Store) Leer(clave string) (string, int64, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    v, ok := s.Datos[clave]
    if !ok {
        return "", 0, false
    }
    return v, s.Versiones[clave], true
}

func (s *ServicioQuorum) Leer(args ArgsLectura, resp *RespLectura) error {
    valor, ts, ok := s.Store.Leer(args.Clave)
    resp.Valor = valor
    resp.Timestamp = ts
    resp.Encontrado = ok
    resp.NodoID = s.NodoID
    return nil
}
```


Go: Leer con quórum y última versión

El cliente lee de R nodos, se queda con la versión más reciente, y si detecta réplicas desactualizadas las repara (read-repair):

```
func CoordinarLectura(clave string, pares []string, r int) (string, int64, bool) {
    mejorValor := ""
    mejorTs := int64(-1)
    respuestas := 0
    encontrado := false

    type respNodo struct {
        par string
        resp RespLectura
    }

    var resultados []respNodo
    // sigue
```

Go: Leer con quórum y última versión

```
for _, p := range pares {
    cliente, err := rpc.Dial("tcp", p)
    if err != nil {
        continue
    }
    var resp RespLectura
    if cliente.Call("ServicioQuorum.Leer", ArgsLectura{clave}, &resp) == nil {
        respuestas++
        if resp.Encontrado {
            encontrado = true
            if resp.Timestamp > mejorTs {
                mejorTs = resp.Timestamp
                mejorValor = resp.Valor
            }
        }
        resultados = append(resultados, respNodo{par: p, resp: resp})
    }
    cliente.Close()
}
```

Go: Leer con quórum y última versión

```
// Read-repair: sincronizar réplicas con timestamp menor
if encontrado && mejorTs > 0 {
    for _, rn := range resultados {
        if rn.resp.Encontrado && rn.resp.Timestamp < mejorTs {
            c, err := rpc.Dial("tcp", rn.par)
            if err == nil {
                var rs RespEscritura
                c.Call("ServicioQuorum.Sincronizar",
                    ArgsEscritura{Clave: clave, Valor: mejorValor, Timestamp: mejorTs}, &rs)
                c.Close()
            }
        }
    }
}

return mejorValor, mejorTs, encontrado && respuestas >= r
}
```

Read-Repair

Cuando una lectura con quórum descubre que algunas réplicas tienen datos desactualizados, las actualiza en el momento:

```
func (s *Store) Sincronizar(clave, valor string, timestamp int64) bool {
    return s.Escribir(clave, valor, timestamp) //
}

func (s *ServicioQuorum) Sincronizar(args ArgsEscritura, resp *RespEscritura) error {
    s.Store.Sincronizar(args.Clave, args.Valor, args.Timestamp)
    resp.Exito = true
    resp.NodoID = s.NodoID
    return nil
}
```

Checkpoint: Read-repair acerca la consistencia eventual a consistencia fuerte sin pagar el costo de escrituras síncronas. Cassandra lo usa como mecanismo principal de reparación. `Store.Sincronizar` reutiliza `Store.Escribir` (ver "Go: Escribir con quórum")

PARTE II: Sharding

Sharding: concepto

- **Definición:** particionar los datos horizontalmente entre múltiples nodos.
- **Objetivos:**
 - **Escalabilidad horizontal:** agregar nodos = más capacidad.
 - **Paralelismo:** consultas en distintos shards se ejecutan en paralelo.
 - **Manejo de volumen:** superar límites de un solo nodo.

Mientras la replicación da tolerancia a fallos, el sharding da escala. Se complementan: cada shard puede replicarse.

Estrategias de sharding

Estrategia	Cómo divide	Ventaja	Desventaja
Por rango	A-M shard 1, N-Z shard 2	Consultas de rango eficientes	Hot spots posibles
Por hash	<code>hash(clave) % N</code>	Distribución uniforme	Consultas de rango difíciles
Por entidad	Usuarios en un shard, productos en otro	Aislamiento natural	Consultas cross-shard

Consistent Hashing

- Ya lo vimos en **DHT Chord**: anillo lógico $0-2^{32}$, cada clave se asigna al nodo más cercano en sentido horario.
- Al agregar o quitar un nodo, solo se reasignan las claves **adyacentes**.
- **Nodos virtuales**: cada nodo físico aparece múltiples veces en el anillo para mejor distribución.

```
Anillo:  [Nodo A] ---- [Nodo B] ---- [Nodo C] ---- [Nodo A]
           |               |
          clave 5         clave 42
```

Checkpoint: Consistent hashing es la misma técnica que Chord, pero aplicada a particionamiento de datos. Dynamo, Cassandra y Discord lo usan.

Sharding dinámico

- **Resharding automático**: redistribuir datos cuando la carga cambia.
- **Split**: un shard grande se divide en dos.
- **Merge**: shards pequeños se fusionan.
- **Migración en caliente**: mover datos sin interrumpir el servicio.

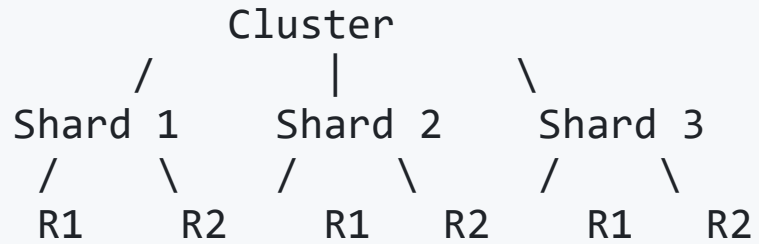
Shard A (lleno)		Shard A1	Shard A2
[1..100]	→	[1..50]	[51..100]

Anti-patrones de sharding

- **Sharding prematuro**: complejidad innecesaria antes de que el volumen lo justifique.
- **Clave de sharding inadecuada**: genera hot spots o consultas cross-shard excesivas.
- **Ignorar patrones de acceso**: no considerar cómo se usarán los datos.
- **Joins cross-shard**: consultas que cruzan múltiples shards son costosas.

Checkpoint: La mayoría de las aplicaciones no necesitan sharding hasta cientos de GB o miles de req/s. Antes de shardear, optimizar consultas y cachear.

Integración: Sharding + Replicación



- Cada shard almacena un subconjunto de datos.
- Cada shard se replica (N, W, R) para tolerancia a fallos.
- Gossip mantiene la membresía de todos los nodos.

Práctica: sd-datastore (PG 4)

Objetivo: Nodo que combina Gossip (membresía) + KV store replicado con quórum.

Servicios RPC:

- `ServicioGossip.Intercambiar` → anti-entropía (reutiliza de PG 3)
- `ServicioQuorum.Escribir` → escritura con quórum (W)
- `ServicioQuorum.Leer` → lectura con quórum (R)
- `ServicioQuorum.Sincronizar` → read-repair

Endpoints HTTP:

- `GET /estado` → membresía + configuración quórum
- `PUT /datos/:clave` → escribir (quórum W)
- `GET /datos/:clave` → leer (quórum R)

Recursos

- **Tanenbaum & Van Steen** cap. 6 (Replicación) y 5.2 (Consistent Hashing)
- **Dynamo**: DeCandia et al. 2007 — origen del modelo (N, W, R)
- **Cassandra**: Lakshman & Malik 2009 — quórum + read-repair + gossip
- **PACELC**: Abadi 2012 — extensión del teorema CAP
- **Brewer**: CAP Twelve Years Later, 2012 — aclaración del teorema CAP
- **Designing Data-Intensive Applications**: Kleppmann 2017 — cap. 5 y 6

¿Preguntas?

Replicación y Sharding en Sistemas Distribuidos